

Stacker: An Experimental Framework for Attention Allocation in Multi-Agent Chat

Florian Noël

2026-04-09 — sketch — CC-BY-4.0

Abstract

This paper describes an experimental framework, not a theory. All claims are hypotheses to be tested through implementation.

We propose Stacker, an experimental coordination mechanism for multi-agent conversational systems. The core design constraint: agents cannot promote themselves into the conversation — they can only amplify other agents' contributions. User attention is treated as a scarce resource allocated through agent interaction rather than central orchestration. Whether this constraint produces useful emergent behavior (such as coalition formation or relevance filtering) is an open empirical question. The framework is inspired by Global Workspace Theory (Baars) but makes no claim of equivalence. This paper defines the experimental setup, the mechanism to be tested, and the observations to be collected.

1. The Problem

1.1. Origin (from Paper 3)

- Paper 3 builds a chat system with multiple background agents operating against RankeDB
- All agents produce potentially relevant context simultaneously
- Context window is finite — not everything fits
- Conversational turns are sequential — only one contribution at a time
- Paper 3 solves this provisionally with heuristics
- This paper asks: is there a mechanism that scales better?

1.2. User Attention as Scarce Resource

- Context window = finite token budget shared between conversation history, agent injections, retrieved knowledge
- Every token consumed by an agent injection is unavailable for other uses
- Every conversational turn taken by an agent is not available for the user or other agents
- This is a resource allocation problem, not a scheduling problem

1.3. The Inversion

- RAG = pull: user query triggers retrieval, system selects context
- Stacker = push: agents proactively offer context, mechanism selects what enters
- Whether push outperforms pull is an empirical question, not an axiom

2. The Mechanism

2.1. Two Channels

- **Silent injection** (low threshold): agent injects context into chat history without taking a conversational turn. Shapes reasoning proactively. User doesn't see it directly.
- **Wortmeldung** (high threshold): agent competes for a visible conversational turn. Higher cost, higher impact.

2.2. The Core Constraint

Agents cannot promote themselves. They can only amplify others.

- An agent that finds relevant context cannot push it into the conversation directly
- It can only signal support for another agent's contribution
- Entry into the conversation requires external validation, not self-assertion
- This constraint is structural, not a rule — the API doesn't offer self-promotion

2.3. What This Might Produce

All of the following are hypotheses, not predictions:

- **Coalition formation:** agents with coherent, mutually reinforcing contributions might naturally cluster and gain access together. Or they might not.
- **Relevance filtering:** low-value contributions might get filtered out because no other agent amplifies them. Or the mechanism might be too noisy.
- **Gaming resistance:** self-promotion is impossible by design. But agents might learn to trade amplification ("I boost you, you boost me"). Whether this is a problem or a feature is unclear.
- **Conversational attention:** contributions that enter the shared context become available to all agents and the reasoning model — analogous to Baars' "conscious broadcast". Whether this analogy is useful or misleading will be revealed by experiment.

TODO — Reading for §2 (retrieval selection in existing GraphRAG systems):

PDF1 §5 is essential. It gives the exact positioning for Stacker: every published system uses a selector; RankeDB removes it.

Selector-based systems (each is a different selector model):

- **Microsoft GraphRAG** (Edge et al. April 2024). User selects local / global / DRIFT mode; deterministic pipeline. medium.com/@zilliz_learn/graphrag-explained_read.pdf. **Priority: M.**
- **Neo4j ToolsRetriever (August 2025). Closest published parallel to Stacker.** Registers multiple retrievers as "tools" and uses an LLM to dynamically select which to invoke per query. **Key contrast: Neo4j delegates selection to an LLM; RankeDB removes the selector entirely.** neo4j.com/blog/developer/introducing-toolsretriever-graphrag-python-package_read.pdf; medium.com/neo4j/introducing-toolsretriever-in-the-neo4j-graphrag-python-package_read.pdf. **Priority: H — this is THE comparison.**
- **LlamaIndex Property Graph Index.** Concurrent retrievers (LLMSynonymRetriever, VectorContextRetriever, CypherTemplateRetriever) but **manual configuration, not emergent.** llamaindex.ai/blog/introducing-the-property-graph-index_read.pdf. **Priority: M.**
- **Adaptive RAG (Jeong et al. 2024).** Uses a **classifier** to route queries to different strategies based on complexity. edenai.co/post/the-2025-guide-to-retrieval-augmented-generation-rag_read.pdf. **Priority: M.**
- **RAG-Fusion (Reciprocal Rank Fusion).** Combines results via RRF at the ranking stage. arxiv.org/html/2506.00054v1. *Closest to execution-competition but still ranks after the fact.* [read.pdf](#). **Priority: M.**

PDF1 core positioning verdict for Paper 4 §2 (copy verbatim into §2.2 or §2.3):

"RankeDB's 'Stacker ecology' — where multiple retrieval strategies execute in parallel and compete through actual performance rather than pre-selection — is architecturally distinct. Neo4j's ToolsRetriever delegates selection to an LLM; Adaptive RAG uses a classifier; RAG-Fusion

*combines results via Reciprocal Rank Fusion. **RankeDB removes the selector entirely, letting results compete directly. This ecological competition model has no direct precedent in the published literature, though the concept of ensemble retrieval with competitive ranking touches adjacent territory.***

Priority: H – this framing is the whole point of §2.

3. Inspiration: Global Workspace Theory

Baars (1988) proposed that consciousness is not a central processor but a shared workspace. Specialized unconscious processors compete for broadcast access. Those that succeed become globally available. Selection happens through coalition — coherent signals from multiple processors reinforce each other.

Structural parallels to Stacker:

Global Workspace	Stacker
Workspace	Chat context window
Unconscious processors	Background agents
Broadcast	Wortmeldung
Coalition formation	Mutual amplification
Conscious access	Entry into context

This is an analogy, not a model. We use it as design inspiration. The experiment will show whether the analogy produces useful behavior or breaks down.

TODO — Reading for §3 (Global Workspace Theory and cognitive-architecture crossovers):

- **Baars, B.J. (1988). “A Cognitive Theory of Consciousness.”** Cambridge University Press. Already cited. **Read chapters on coalition formation and broadcast selection before finalizing §3. Priority: H.**
- **“Graph-Native Cognitive Memory for AI Agents: Formal Belief Revision Semantics for Versioned Memory Architectures”** (arXiv 2603.17244, 2025-2026). Applies AGM belief revision to Neo4j AI memory. **Closest published crossover between cognitive architecture and knowledge graph memory — may influence Paper 4’s framing.** [read.pdf](#). **Priority: H.**
- **Carneades argumentation framework.** Implements varying proof standards per statement — analogous to per-agent conviction/weight in Stacker. SAGE 10.1080/19462166.2012.661766. [read.pdf](#). **Priority: L.**
- **ASPIC+ framework.** Tracks strict and defeasible inference rules with three attack types. [homepages.abdn.ac.uk/n.oren/pages/TAFA-17/papers/TAFA-17_paper_15.pdf](#). [read.pdf](#). **Priority: L.**
- **AKReF (2025, arXiv 2506.00713).** Argumentation knowledge graphs from text using ASPIC+. Heterogeneous graphs with argument nodes and attack/support edges. **Relevant to modeling agent amplification as an argumentation graph.** [read.pdf](#). **Priority: M.**

- **ICLR 2026 MemAgents Workshop.** Calls explicitly for research on multi-agent memory coordination. OpenReview U51WxL382H. **Positions Paper 4 at the 2026 research frontier.** read_2.pdf. **Priority: M.**

4. What We Don't Know

This section is the most important part of the paper.

- Does the no-self-promotion constraint actually produce emergent order, or just deadlock?
- Is mutual amplification sufficient for selection, or does it need additional mechanisms?
- What happens with only 2 agents vs. 5 vs. 20?
- Does the mechanism degrade gracefully under load or collapse suddenly?
- Is there a minimum viable version simpler than the full design?
- How does the mechanism handle rapid topic shifts?
- What does “relevance” even mean in this context, and can agents estimate it?
- Does the system converge to stable patterns or remain chaotic?
- Is the Baars analogy generative (leads to useful design decisions) or decorative?
- Would a simple scoring heuristic from Paper 3 outperform all of this?

5. Experimental Design

5.1. Prerequisites

- Paper 3 chat engine with multiple agents (infrastructure)
- At least 3 heterogeneous agents (researcher, verifier, memory filler)
- Instrumented context window (measure utilization per agent, per turn)
- Logging of all amplification signals between agents

5.2. Baselines

1. **Uncoordinated:** all agents inject freely (expected: overflow, noise)
2. **Round-robin:** agents take turns (expected: fair but irrelevant injections)
3. **Central orchestrator:** single LLM decides who speaks (expected: reasonable but bottleneck)
4. **Stacker:** mutual-amplification mechanism (expected: unknown — that’s the point)

5.3. Observations

- Context utilization: what % of injected tokens was used by the reasoning model?
- Response quality: human evaluation, with and without Stacker
- Agent diversity: does the mechanism favor certain agents over time?
- Emergence: do stable patterns (coalitions, roles, rhythms) appear?
- Failure modes: when and how does the mechanism break?

5.4. What Counts as Success

- Stacker outperforms uncoordinated and round-robin on response quality
- Stacker matches or approaches central orchestrator without requiring a central intelligence
- Observable emergent structure (coalitions, turn-taking patterns) that wasn’t explicitly designed
- If none of these occur, the experiment has still produced useful negative results

6. Conclusion

Stacker is an experiment, not a theory. The hypothesis is that a simple structural constraint — agents can only amplify others, never themselves — might produce useful self-organization in multi-agent

chat. The inspiration is economic (attention as scarce resource) and cognitive (Global Workspace Theory). Whether the mechanism works is unknown. This paper defines what “works” means and how to test it.

The only claim we make with confidence: the coordination problem from Paper 3 is real and will get worse as the number of agents grows. Whether Stacker solves it, or whether a simpler mechanism suffices, is the question this experiment answers.

References

- Baars, B.J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.
- *Papers 1–3 (RankerDB, Workers, Applications)*
- *TBD — multi-agent coordination, mechanism design, attention allocation*

Sketch v0.1 — April 2026. This paper cannot be written until Papers 1–3 are implemented and experimental data exists.